



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ИНФОРМАЦИОННЫЕ СИСТЕМЫ В ХТ

Лекция 14. Языки программирования

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

-  Рейтинги востребованности ЯП
-  Парадигмы программирования
-  Объектно-ориентированное программирование
-  Функциональное программирование
-  Обзор популярных языков программирования

Индекс TIOBE (tiobe.com)

Mar 2025	Mar 2024	Change	Programming Language		Ratings	Change
1	1			Python	23.85%	+8.22%
2	3	▲		C++	11.08%	+0.37%
3	4	▲		Java	10.36%	+1.41%
4	2	▼		C	9.53%	-1.64%
5	5			C#	4.87%	-2.67%
6	6			JavaScript	3.46%	+0.08%
7	8	▲		Go	2.78%	+1.22%
8	7	▼		SQL	2.57%	+0.65%
9	10	▲		Visual Basic	2.52%	+1.09%
10	15	▲		Delphi/Object Pascal	2.15%	+0.94%
11	14	▲		Fortran	1.70%	+0.48%
12	9	▼		Scratch	1.66%	+0.21%
13	12	▼		PHP	1.48%	+0.16%
14	17	▲		Rust	1.23%	+0.20%
15	13	▼		MATLAB	0.98%	-0.26%
16	21	▲		R	0.94%	+0.13%
17	11	▼		Assembly language	0.87%	-0.52%
18	24	▲		Ada	0.85%	+0.10%
19	19			Kotlin	0.85%	-0.11%
20	20			COBOL	0.84%	+0.01%

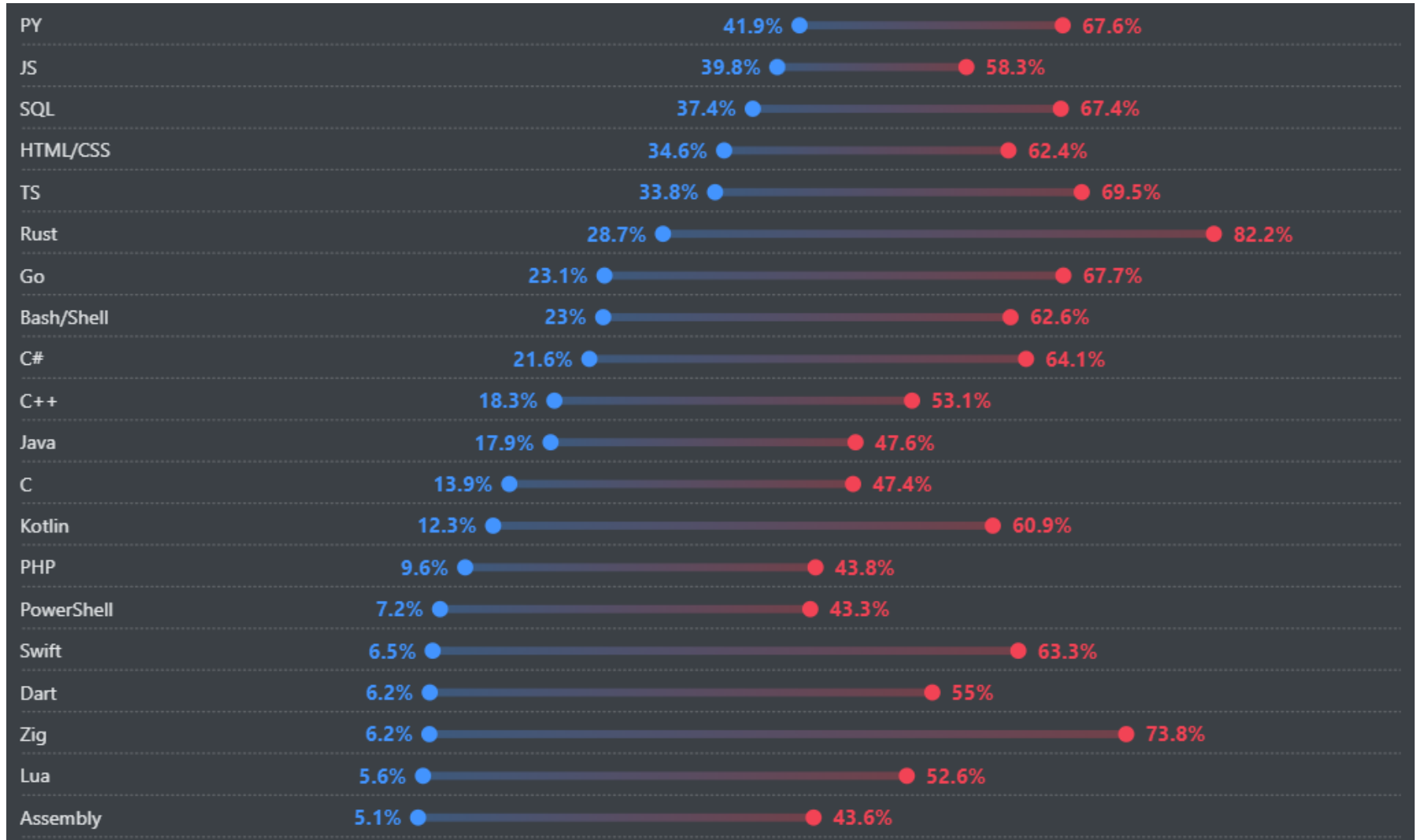
Индекс PYPL (pypl.github.io)

PopularitY of Programming Language

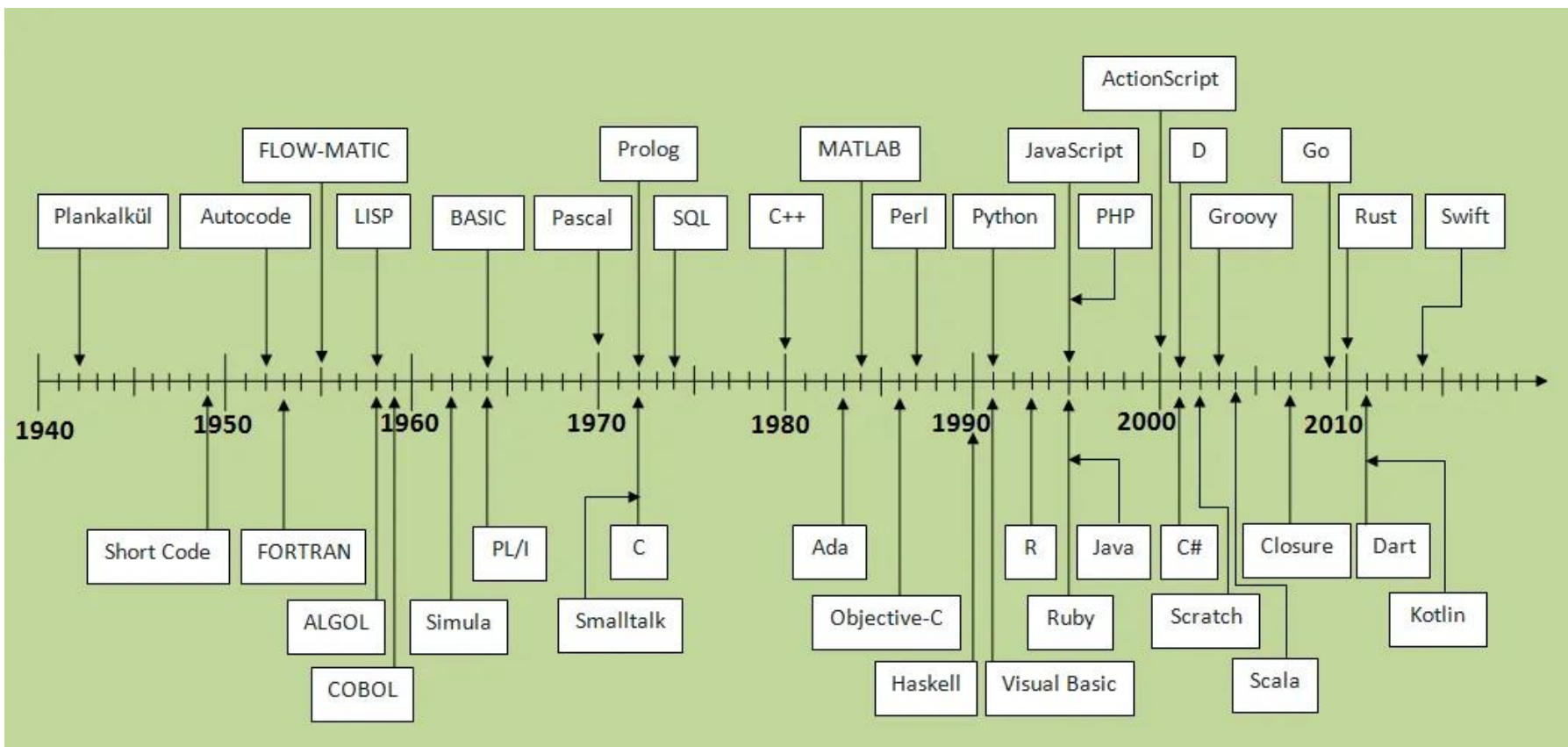
Rank	Change	Language	Share	1-year trend
1		Python	30.27 %	+1.8 %
2		Java	14.89 %	-0.9 %
3		JavaScript	7.78 %	-0.9 %
4	↑	C/C++	7.12 %	+0.6 %
5	↓	C#	6.11 %	-0.6 %
6		R	4.54 %	-0.1 %
7		PHP	3.74 %	-0.7 %
8	↑↑	Rust	3.14 %	+0.6 %
9	↓	TypeScript	2.78 %	-0.1 %
10	↑	Objective-C	2.74 %	+0.3 %
11	↓↓	Swift	2.44 %	-0.3 %
12		Go	2.06 %	-0.2 %
13		Kotlin	1.9 %	+0.0 %
14		Matlab	1.68 %	+0.1 %
15	↑	Ada	1.33 %	+0.3 %

16	↓	Dart	1.03 %	-0.0 %
17		Ruby	1.01 %	+0.0 %
18	↑↑	Lua	0.96 %	+0.2 %
19	↓	Powershell	0.95 %	+0.0 %
20	↓	VBA	0.9 %	+0.0 %
21		Scala	0.59 %	-0.0 %
22		Abap	0.51 %	-0.0 %
23		Visual Basic	0.43 %	-0.1 %
24		Julia	0.39 %	+0.1 %
25		Groovy	0.17 %	-0.1 %
26	↑↑↑↑	Zig	0.17 %	+0.1 %
27	↓	Haskell	0.15 %	-0.0 %
28		Cobol	0.11 %	-0.0 %
29	↓↓	Perl	0.07 %	-0.1 %
30	↓	Delphi/Pascal	0.03 %	-0.1 %

Индекс StackOverflow (survey.stackoverflow.co)



История появления ЯП



Парадигмы программирования

Императивная

Программа – последовательность команд (операций, изменяющих состояние программы)

- ◉ Процедурная
 - Структурная (FORTRAN, BASIC, Pascal, C)
- ◉ Объектно-ориентированная
 - Основанная на классах (C++, Perl, Python, Java, PHP, Ruby, C#)
 - Основанная на прототипах (JavaScript)

Декларативная

Программа – описание логики решения задачи без указания определенной последовательности действий (например, SQL)

- ◉ Функциональная (Lisp, Haskell, JavaScript, Java 8+)
- ◉ Логическая
- ◉ Реактивная

Другие...

Объектно-ориентированное программирование

- ☎ Метод программирования, основанный на:
 - ☉ понятии объекта, как формально описанной совокупности данных и процедур;
 - ☉ реализации алгоритмов, как описания взаимодействия объектов между собой
 - ☉ понятии класса объектов и описании иерархий классов*
*(кроме ООП языков, основанных на прототипах)
- ☎ Лучшие практики разработки программного обеспечения
 - ☉ Использование ООП позволяет создать расширяемое, масштабируемое и легкоуправляемое ПО
- ☎ Тесная связь модели компьютерной системы с объектами реального мира

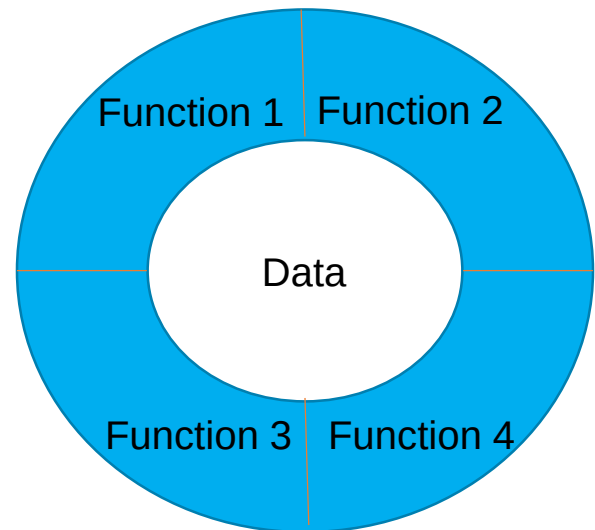
Понятие объекта

- Представим сущность из реального мира, например, компьютер
- С компьютером связаны:
 - Набор данных, таких как объем памяти, количество процессорных ядер, параметры операционной системы и т.д.
 - Набор действий, которые могут быть выполнены с компьютером, такие как включение, выключение, добавление модулей памяти и т.д.
- Сущность реального мира может быть смоделирована в рамках языка программирования, такая модель называется объектом



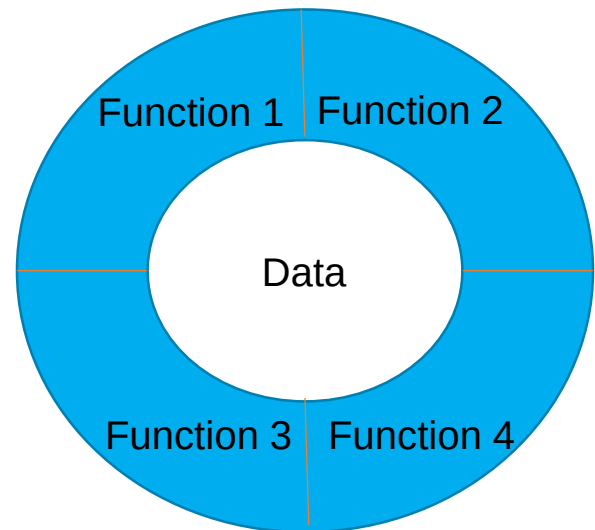
Что такое объект? (1/2)

- 🌀 Объект – модель сущности, которая состоит из данных и управляющих ими функций
 - 🌀 Объекты – это материальные сущности, бизнес сущности или абстракции
 - 🌀 Свойства или состояние сущности представлены данными объекта
 - 🌀 Функции объекта моделируют действия, которые могут быть выполнены сущностью /с сущностью
 - 🌀 Совокупность функций объекта называют его поведением



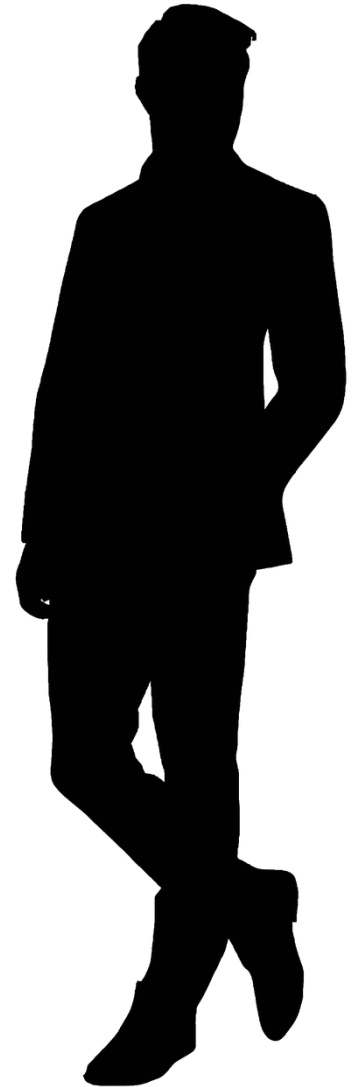
Что такое объект? (2/2)

- ☉ Данные и реализации функций инкапсулируются в объекте
 - ☉ Инкапсуляция означает замкнутость или скрытость
 - ☉ Доступ к данным для чтения или изменения осуществляется только посредством функций этого объекта
 - ☉ Детали реализации функций скрыты от вызывающей стороны



Пример объекта

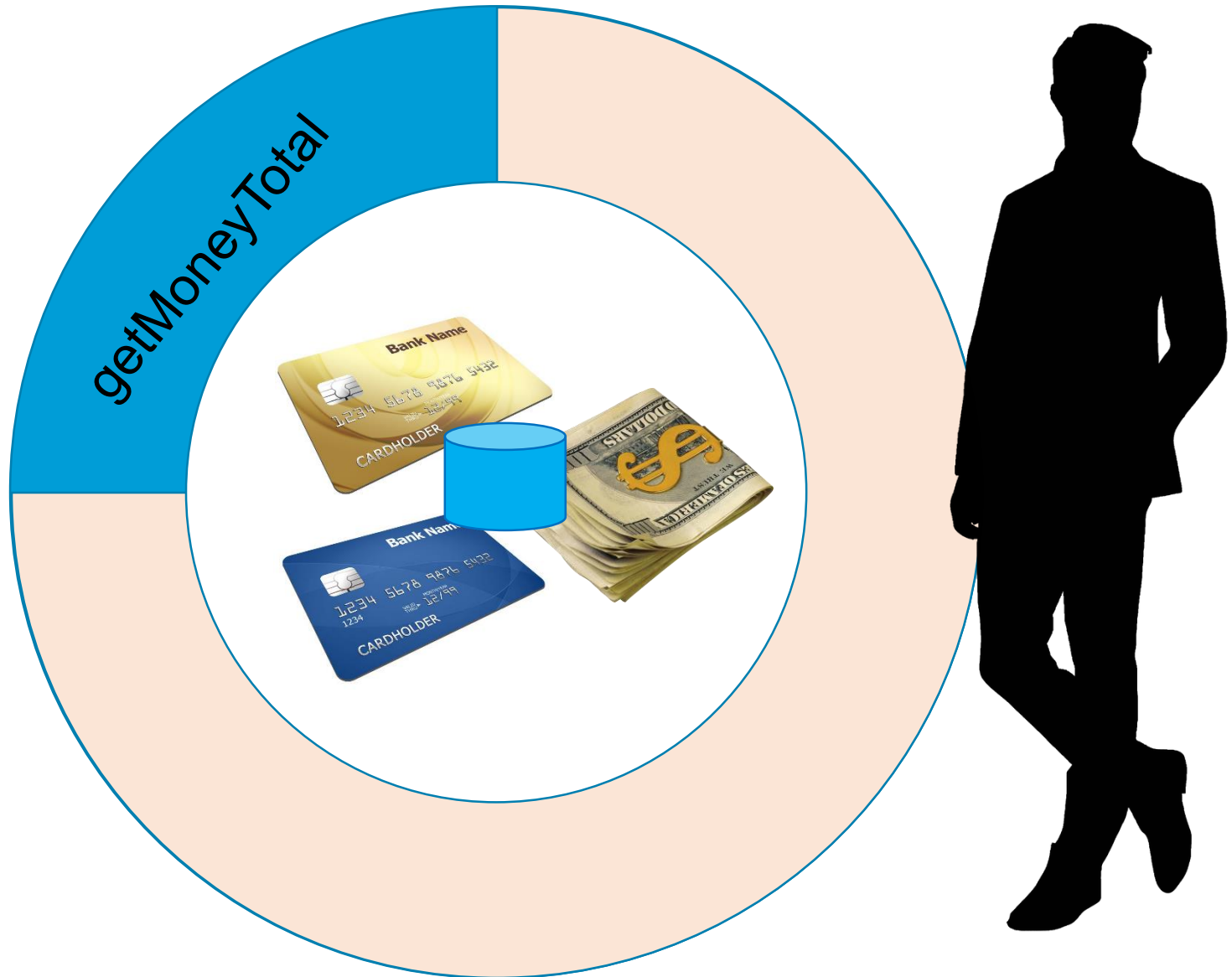
- 📡 Сущность - человек
- 📡 Свойства: Имя, Фамилия, Возраст...
- 📡 Поведение: Ходит, Говорит, Работает...
- 📡 Как узнать, сколько у человека при себе денег?



Процедурный подход

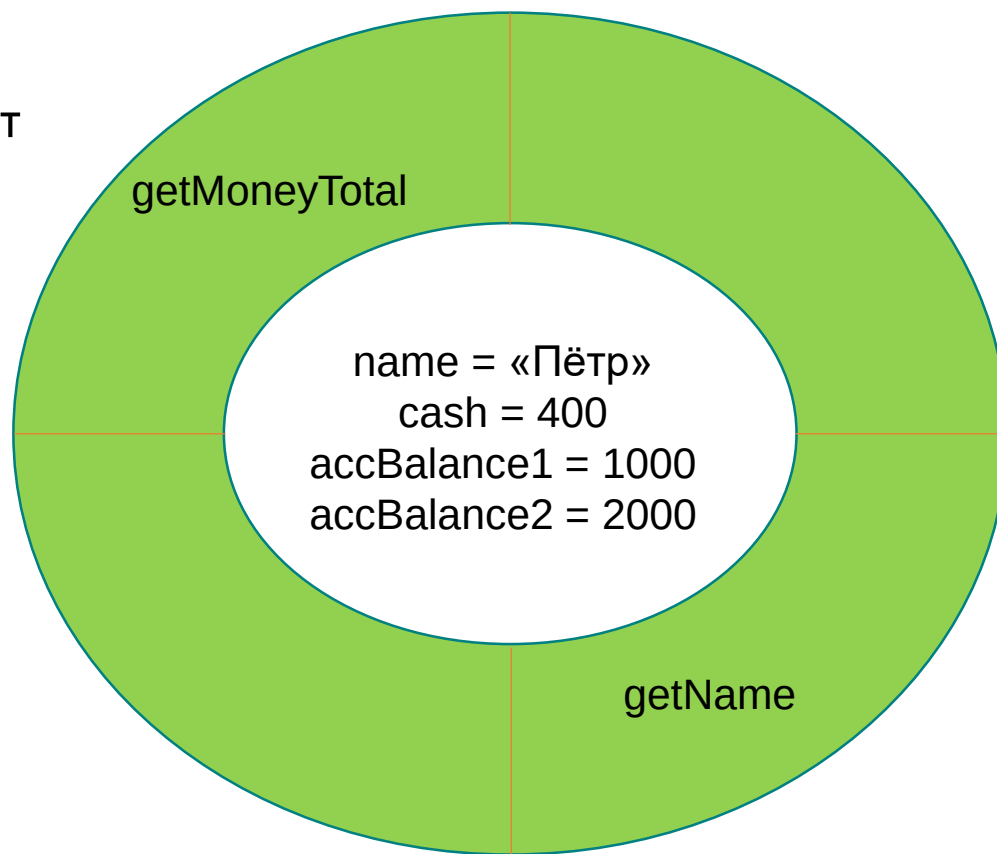


Объектно-ориентированный подход



Объект: Человек

- Данные инкапсулированы внутри объекта, прямой доступ к ним извне объекта закрыт
 - Если мы хотим узнать значение поля name, мы должны использовать функцию getName
- Функции объекта позволяют получить его состояние или изменить его
- При проектировании приложения мы моделируем значимое для нас поведение объекта в виде бизнес-функций
 - getMoneyTotal



Сообщения и методы (1/2)

Сообщение

- ⦿ Сообщение - это запрос на выполнение объектом действия (или предоставление им услуги)
- ⦿ Обязанность объекта - выбрать процедуру, соответствующую сообщению, и выполнить ее код

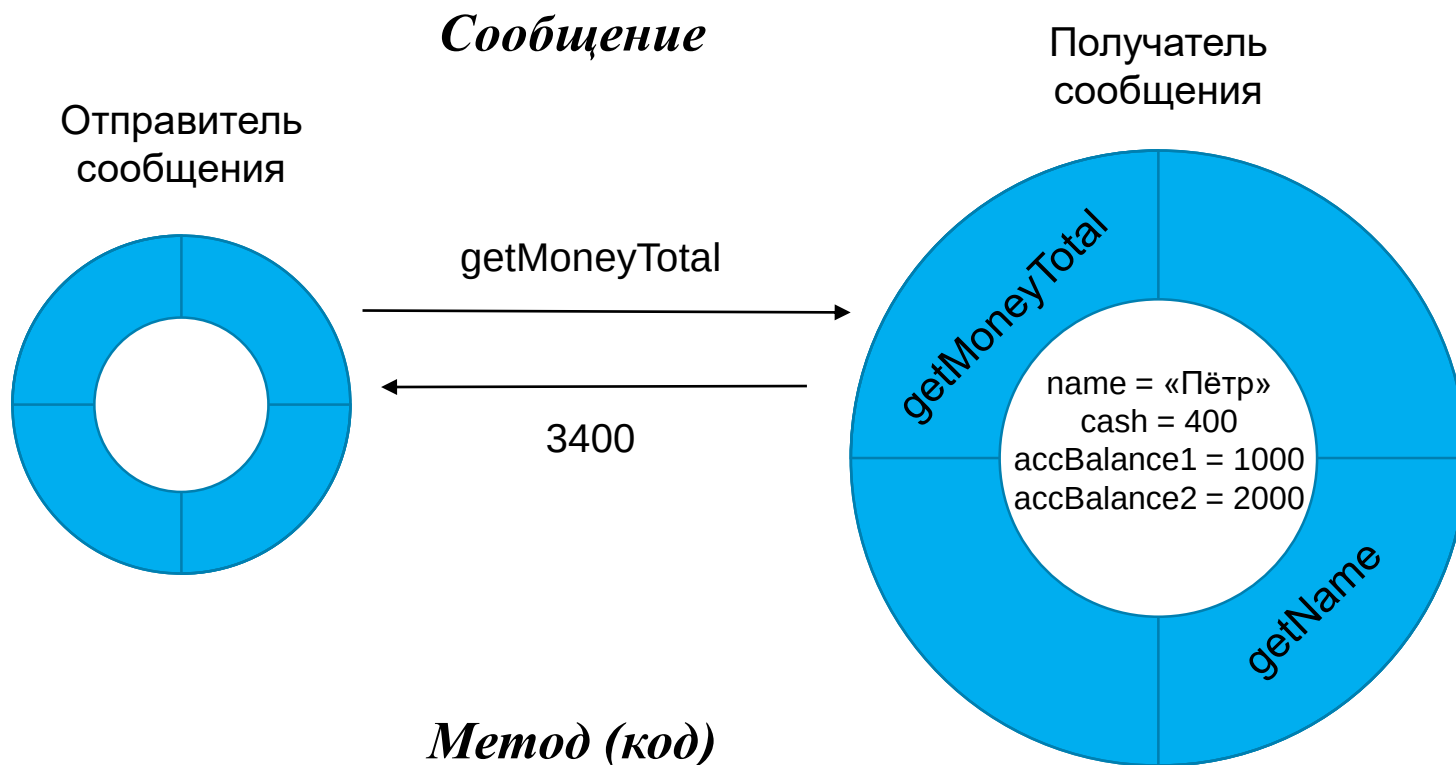
Метод

- ⦿ Метод - это реализация действия, запрошенного в сообщении
- ⦿ Метод – процедура, являющаяся частью поведения объекта, содержащая код, реализующий алгоритм действия

Динамический вызов / обработка сообщения

- ⦿ Выбор метода, который будет вызван, определяется во время выполнения

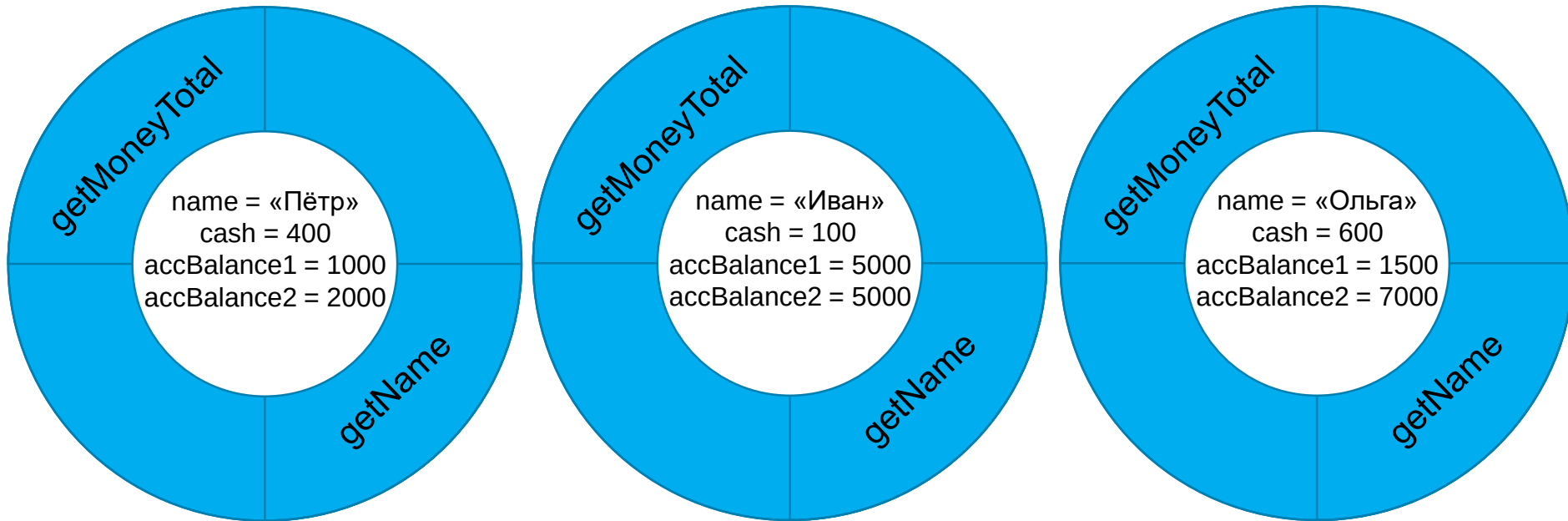
Сообщения и методы (2/2)



Доступ к данным

- ☎ Доступ к данным, хранящимся в объектах, другие объекты могут получить непосредственно* или посредством отправки сообщений
 - ☉ Прямой доступ нарушает принцип инкапсуляции
- ☎ Рекомендуемый способ – отправка сообщений
 - ☉ Сообщения позволяют разработчикам скрыть реализацию методов и детали внутреннего представления данных
- ☎ Обычно объект предоставляет набор методов, позволяющих получить доступ к его данным
 - ☉ Например, используя сообщение `getMoneyTotal`, мы узнаем количество денег у человека, при этом не зная, как рассчитывается сумма и из каких источников берутся данные для расчета

Объекты и классы



- Три объекта имеют одинаковую структуру данных и поведение
- Значения данных свои для каждого человека
- Переменные, необходимые для хранения данных, одинаковы во всех случаях; объекты отвечают на одни и те же сообщения
 - Все три объекта – экземпляры одного и того же класса объектов

Классы и экземпляры (1/2)

- Класс – множество объектов, представляющих однотипные сущности
 - Яблоко, апельсин, груша – это фрукты
- Экземпляр класса – объект представитель класса
 - Яблоко – представитель класса фрукты
 - Экземпляр и объект в ООП – синонимы



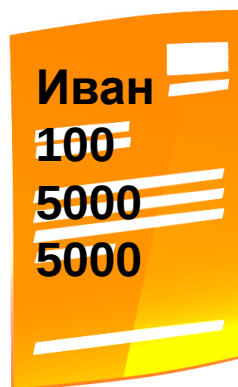
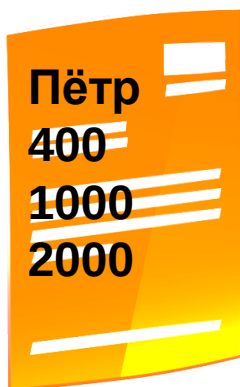
Классы и экземпляры (2/2)

📡 Класс определяет структуру, единую для всех экземпляров этого класса:

- ⦿ Набор полей данных
- ⦿ Набор методов
 - Сообщения, которые может получать экземпляр
 - Реализация обработки сообщений



📡 Экземпляры класса идентичны по структуре, но могут отличаться значениями полей данных

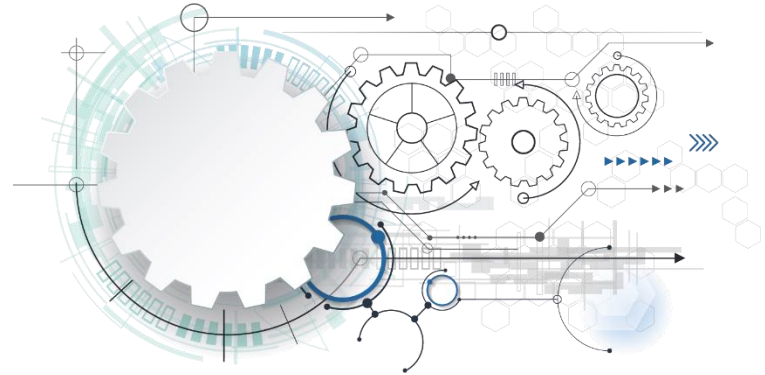


Пример класса «Человек» (Java)

```
public class Person {  
    private String name;  
    private int cash;  
    private int accBalance1;  
    private int accBalance2;  
  
    public String getName() {  
        return name;  
    }  
  
    public int getMoneyTotal() {  
        return cash+accBalance1+accBalance2;  
    }  
}
```

Принципы объектно-ориентированного программирования

- Модульность
- Абстракция
- Наследование



Эти принципы объектно-ориентированного программирования позволяют реализовывать лучшие практики разработки программного обеспечения



Пример полиморфизма (Python)

```
class Animal:
    def speak(self):
        raise NotImplementedError("Метод speak() должен быть переопределен в подклассе")

class Dog(Animal):
    def speak(self):
        return "Гав!"

class Cat(Animal):
    def speak(self):
        return "Мяу!"

class Cow(Animal):
    def speak(self):
        return "Муу!"

def animal_sound(animal: Animal): # Функция, демонстрирующая полиморфизм
    print(animal.speak())

animals = [Dog(), Cat(), Cow()] # Функция, демонстрирующая полиморфизм

for animal in animals: # Вызываем один и тот же метод для разных объектов
    animal_sound(animal)
```

Гав!
Мяу!
Муу!

Функциональное программирование

Чистые функции

- ◉ Функции без побочных эффектов – все данные функция получает через параметры и возвращает явно, выполнение функции не зависит от состояния программы

Неизменяемость данных

- ◉ Вместо изменения данных создаются новые

Функции высшего порядка

- ◉ Функции, которые принимают или возвращают другие функции

Шаблоны

- ◉ Рекурсия вместо циклов
- ◉ Замыкания
- ◉ Каррирование
- ◉ Композиция функций
- ◉ Генераторы и ленивые вычисления

Пример "чистой" функции (Python)

Чистая функция (предсказуема, не изменяет внешний мир)

```
def square(x):  
    return x ** 2
```

Нечистая функция (изменяет внешний список)

```
def add_to_list(item, lst=[]):  
    lst.append(item) # Побочный эффект!  
    return lst
```

Пример неизменяемость (Python)

Плохо (изменяемый список)

```
lst = [1, 2, 3]
```

```
lst.append(4) # Изменяет исходный список
```

Лучше (создание нового списка)

```
new_lst = lst + [4] # Исходный lst не изменяется
```

Использование кортежей (tuple) – неизменяемых аналогов списков

```
point = (2, 3) # Нельзя изменить после создания
```

Пример функции высшего порядка (Python)

Функция, возвращающая функцию

```
def make_adder(n):  
    def adder(x):  
        return x + n  
    return adder
```

```
add5 = make_adder(5)  
print(add5(3))  # 8
```

Функция, принимающая функцию как аргумент

```
def apply(func, value):  
    return func(value)
```

```
print(apply(lambda x: x * 2, 7))  # 14
```

Пример рекурсия вместо циклов (Python)

Рекурсивный факториал

```
def factorial(n):
```

```
    return 1 if n == 0 else n * factorial(n - 1)
```

Хвостовая рекурсия (оптимизированный вариант)

```
def factorial_tail(n, acc=1):
```

```
    return acc if n == 0 else factorial_tail(n-1, acc*n)
```

Пример замыкания (Python)

- 🌀 Функции, "запоминают" окружение, в котором они были созданы

```
def counter():  
    count = 0  
    def increment():  
        nonlocal count  
        count += 1  
        return count  
    return increment  
  
c = counter()  
print(c(), c(), c()) # 1 2 3
```

Пример каррирование (Python)

- 🌀 Преобразование функции от многих аргументов в последовательность функций одного аргумента

```
def curry_add(a):  
    def add_b(b):  
        return a + b  
    return add_b
```

```
add5 = curry_add(5)  
print(add5(3)) # 8
```

```
# Альтернатива с functools.partial  
from functools import partial  
add5 = partial(lambda a, b: a + b, 5)  
print(add5(3)) # 8
```

Пример ленивые вычисления (Python)

- 🌀 Постепенная генерация значений без загрузки всей последовательности в память

```
def fibonacci():
```

```
    a, b = 0, 1
```

```
    while True:
```

```
        yield a
```

```
        a, b = b, a + b
```

```
fib = fibonacci()
```

```
print(next(fib), next(fib), next(fib))  # 0 1 1
```

Интерпретаторы и компиляторы

	Компилятор	Интерпретатор
Принцип работы	Преобразует весь исходный код в машинный код (или в промежуточный код) заранее.	Выполняет программу построчно, переводя каждую инструкцию в машинный код на лету.
Скорость выполнения	Быстрее (код уже оптимизирован и скомпилирован).	Медленнее (каждая строка анализируется и выполняется отдельно).
Запуск программы	Требует отдельного шага компиляции перед запуском.	Можно запускать сразу, без предварительной обработки.
Отладка	Сложнее (ошибки обнаруживаются на этапе компиляции).	Проще (можно увидеть ошибку сразу при выполнении строки).
Кроссплатформенность	Зависит от целевой платформы (нужно компилировать под каждую ОС).	Легче переносится (достаточно иметь интерпретатор для нужной платформы).
Примеры языков	C, C++, Go, Rust, Pascal.	Python, JavaScript (в браузере), Ruby, PHP, Bash.
Использование памяти	Оптимизированный бинарный код обычно потребляет меньше памяти при выполнении.	Требуется больше памяти, так как интерпретатор хранит исходный код и состояние.
Гибкость	Менее гибкий (изменения требуют перекомпиляции).	Более гибкий (можно быстро менять код и сразу запускать).

Языки программирования (1/2)

C/C++

- ◉ Компилируемый, статическая типизация
- ◉ Процедурное, объектно-ориентированное, обобщенное программирование
- ◉ Низкоуровневое и системное программирование, создание высокопроизводительных программ
- ◉ Много библиотек для решения систем линейных, нелинейных, дифференциальных уравнений
- ◉ Высокий порог вхождения



Java

- ◉ Компилируемый в байт-код VM, статическая типизация, автоматическое управление памятью
- ◉ Объектно-ориентированное, обобщенное, функциональное программирование
- ◉ Использует виртуальную машину, переносимость на бинарном уровне
- ◉ Встроенные средства многопоточности и безопасности
- ◉ Создание программ корпоративного уровня, программирование под мобильные устройства и встраиваемую технику
- ◉ Средний порог вхождения



Языки программирования (2/2)

Python

- ◉ Интерпретируемый, динамическая типизация, автоматическое управление памятью
- ◉ Структурное, объектно-ориентированное, функциональное, императивное и аспектно-ориентированное программирование
- ◉ Богатый набор библиотек для работы с данными, включая TensorFlow
- ◉ Создание средств анализа больших данных, машинного обучения, искусственного интеллекта
- ◉ Низкий порог вхождения



JavaScript

- ◉ Интерпретируемый, динамическая типизация, автоматическое управление памятью
- ◉ Процедурное, функциональное, объектно-ориентированное, программирование
- ◉ Веб-программирование, написание сценариев
- ◉ Каркас Node.js
- ◉ Низкий порог вхождения



Go (Golang)

- ◉ Компилируемый, статическая типизация
- ◉ Процедурное, функциональное, объектно-ориентированное, программирование
- ◉ Призван заменить C/C++
- ◉ Средний порог вхождения



Python

- 🔊 **Тип языка:** Высокоуровневый, интерпретируемый, с динамической типизацией.
- 🔊 **Парадигмы:** Мультипарадигменный (ООП, функциональное, императивное, процедурное).
- 🔊 **Синтаксис:** Простой и читаемый («как псевдокод»), с отступами вместо скобок.
- 🔊 **Скорость:** Медленнее C/Java, но с оптимизациями (Cython, PyPy) и библиотеками (NumPy).
- 🔊 **Применение:**
 - 🔊 Веб (Django, Flask),
 - 🔊 Data Science (Pandas, NumPy, TensorFlow),
 - 🔊 Автоматизация, скрипты,
 - 🔊 ИИ/ML (PyTorch, scikit-learn).
- 🔊 **Плюсы:**
 - 🔊 Лёгкость обучения,
 - 🔊 Большое сообщество и библиотеки,
 - 🔊 Кроссплатформенность.
- 🔊 **Минусы:**
 - 🔊 Низкая скорость в CPU-bound задачах,
 - 🔊 Глобальная блокировка интерпретатора (GIL) в многопоточности.
- 🔊 Python идеален для быстрой разработки и сложных проектов с упором на удобство, а не на производительность

Java

- ☞ **Тип языка:** Высокоуровневый, компилируемый в байт-код (JVM), со статической типизацией.
- ☞ **Парадигмы:** ООП (классы, наследование), поддержка функционального стиля (с Java 8+).
- ☞ **Синтаксис:** Чёткий, но многословный (требует объявления типов, точек с запятой, фигурных скобок).
- ☞ **Скорость:** Быстрее Python, но медленнее C/C++ за счёт JVM и сборщика мусора (GC).
- ☞ **Применение:**
 - Корпоративные приложения (Spring, Jakarta EE),
 - Android-разработка (до Kotlin),
 - Big Data (Hadoop, Spark),
 - Микросервисы и облачные решения.
- ☞ **Плюсы:**
 - Кроссплатформенность («Write Once, Run Anywhere»),
 - Мощная экосистема (Maven, Gradle, IntelliJ IDEA),
 - Надёжность и безопасность (статика + JVM).
- ☞ **Минусы:**
 - Большой объём кода (boilerplate),
 - Нужна JVM для работы,
 - Менее гибкий, чем Python/JavaScript.
- ☞ **Идеален для:**
 - Масштабируемых enterprise-решений,
 - Проектов, где важна стабильность и производительность.

JavaScript

- 🧐 **Тип языка:** Высокоуровневый, интерпретируемый (JIT-компиляция в браузерах), с динамической типизацией.
- 🧐 **Парадигмы:** Мультипарадигменный (ООП, функциональный, императивный, событийно-ориентированный).
- 🧐 **Синтаксис:** С-подобный (фигурные скобки, точки с запятой опциональны), гибкий, но с «подводными камнями»
- 🧐 **Скорость:** Достаточно быстрый благодаря движкам (V8, SpiderMonkey), но медленнее Java/C++.
- 🧐 **Применение:**
 - 🕒 Веб-фронтенд (React, Vue, Angular),
 - 🕒 Серверная разработка (Node.js, Express),
 - 🕒 Мобильные приложения (React Native),
 - 🕒 Десктоп (Electron).
- 🧐 **Плюсы:**
 - 🕒 Запускается везде (браузеры, серверы, мобильные устройства),
 - 🕒 Огромное сообщество и экосистема (npm — крупнейший репозиторий пакетов),
 - 🕒 Гибкость и простота старта.
- 🧐 **Минусы:**
 - 🕒 Слабая типизация (может приводить к ошибкам),
 - 🕒 «Странности»
 - 🕒 Зависимость от среды выполнения (браузер, Node.js).
- 🧐 **Идеален для:**
 - 🕒 Веб-разработки (фронтенд + бэкэнд),
 - 🕒 Быстрого прототипирования,
 - 🕒 Кросс-платформенных решений.

Итоги

В данной лекции были рассмотрены следующие темы:

-  Рейтинги востребованности ЯП
-  Парадигмы программирования
-  Объектно-ориентированное программирование
-  Функциональное программирование
-  Обзор популярных языков программирования